```
# spam_detector.py
#
# This is the main script for my spam email
detector project.
# The idea is simple: give it a message,
and it tells you if it thinks
# the message is spam or not (ham =
normal/legit message).
#
# Steps I followed:
# 1. Load the dataset (label + text)
# 2. Turn the text into numbers using TF-
IDF (a computer can't read words directly)
# 3. Train a Naive Bayes model on those
numbers
# 4. Check how well it does on messages it
hasn't seen before
# 5. Save the trained model so I don't have
to retrain it every time
#
# I used Naive Bayes because it's one of
the algorithms we covered in class
# for text classification, and it's known
to work decently well for spam
# detection even though it's fairly simple.

import argparse
import os
import joblib
import pandas as pd
```

```
from sklearn.model_selection import
train_test_split
from sklearn.feature_extraction.text import
TfidfVectorizer
from sklearn.naive_bayes import
MultinomialNB
from sklearn.metrics import accuracy_score,
precision_score, recall_score, f1_score,
confusion_matrix

DATASET_PATH = os.path.join("dataset",
"spam_dataset.csv")
MODEL_PATH = os.path.join("model",
"spam_model.joblib")
VECTORIZER_PATH = os.path.join("model",
"vectorizer.joblib")


def load_dataset():
    data = pd.read_csv(DATASET_PATH)
    # dropping any rows that might have
missing values, just to be safe
    data = data.dropna(subset=["text",
"label"])
    return data


def train_model():
    data = load_dataset()

    messages = data["text"]
    labels = data["label"]

    # splitting into training and testing
```

```
sets so I can check performance
    # on messages the model was not trained
on
    X_train, X_test, y_train, y_test =
train_test_split(
        messages, labels, test_size=0.2,
random_state=42
    )

    # TF-IDF turns each message into a
vector of numbers based on word
    # importance. I added ngram_range=(1,2)
after noticing single words
    # weren't catching phrases like "act
now" or "click here" very well.
    vectorizer =
TfidfVectorizer(stop_words="english",
ngram_range=(1, 2))
    X_train_vectors =
vectorizer.fit_transform(X_train)
    X_test_vectors =
vectorizer.transform(X_test)

    model = MultinomialNB()
    model.fit(X_train_vectors, y_train)

    predictions =
model.predict(X_test_vectors)

    # printing out some basic metrics to
see how the model did
    print("Accuracy:",
round(accuracy_score(y_test, predictions),
3))
```

```
    print("Precision:",
round(precision_score(y_test, predictions,
pos_label="spam"), 3))
    print("Recall:",
round(recall_score(y_test, predictions,
pos_label="spam"), 3))
    print("F1 score:",
round(f1_score(y_test, predictions,
pos_label="spam"), 3))
    print("Confusion matrix (rows = actual,
columns = predicted):")
    print(confusion_matrix(y_test,
predictions, labels=["ham", "spam"]))

    # saving the model and vectorizer so I
can reuse them later without retraining
    os.makedirs("model", exist_ok=True)
    joblib.dump(model, MODEL_PATH)
    joblib.dump(vectorizer,
VECTORIZER_PATH)
    print("Model saved in the model/
folder")

    return model, vectorizer


def predict_message(message, model=None,
vectorizer=None):
    # if no model was passed in, load the
saved one from disk
    if model is None or vectorizer is None:
        model = joblib.load(MODEL_PATH)
        vectorizer =
joblib.load(VECTORIZER_PATH)
```

```
    message_vector =
vectorizer.transform([message])
    prediction =
model.predict(message_vector)[0]

    print(f"Message: {message}")
    print(f"Prediction: {prediction}")
    return prediction


if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--predict",
type=str, help="check a single message")
    args = parser.parse_args()

    if args.predict:
        predict_message(args.predict)
    else:
        trained_model, trained_vectorizer =
train_model()

        # just testing it on two example
messages to see if it makes sense
        print()
        print("Testing on a couple of
example messages:")
        predict_message("Hey, are we still
on for lunch tomorrow?", trained_model,
trained_vectorizer)
        predict_message("CONGRATULATIONS!
```

```
You have won a free gift card, click here
now", trained_model, trained_vectorizer)
```